

Barrel-Shifter Hamming(16,11) Correction

How the fb2d gadget corrects single-bit errors using only 1 dirty garbage cell, explained from first principles.

Background: what Hamming correction needs to do

A 16-bit Hamming(16,11) SECDED codeword has 11 data bits and 5 parity bits. When a single bit gets flipped, we need to figure out *which* bit and flip it back.

Standard-form Hamming gives us two pieces of information:

- **syndrome** (4 bits): tells us the *position number* (0-15) of the flipped bit. Syndrome 0 means “no error at positions 1-15”.
- **p_all** (1 bit): the overall parity of all 16 bits. Tells us whether an odd number of bits were flipped.

Together these give four cases:

p_all	syndrome	meaning	action
0	0	no error	do nothing
1	0	bit 0 (overall parity) bad	flip bit 0
1	k (1-15)	bit k is bad	flip bit k
0	k (1-15)	double error (detected)	don't correct

So the core task is: **flip exactly one bit of the codeword, at a position determined at runtime by the syndrome.**

The problem: runtime-variable bit position

In fb2d, we can XOR two cells ($x: [H0] \hat{=} [H1]$), but this flips *all* differing bits between two 16-bit values. To flip exactly one bit, we need [H1] to contain a value with exactly one bit set — a **1-hot mask** like 0000000000000100 (for bit 2).

The challenge: the syndrome is computed at runtime, so we can't hardcode which mask to use. We need to *build* the mask $1 \ll \text{syndrome}$ dynamically using only reversible operations.

What “ $1 \ll \text{syndrome}$ ” means

$\ll$ is the **left shift** operator. $1 \ll k$ means “start with the number 1 (binary 0000000000000001) and shift it left by k positions”:

```
1 << 0 = 0000000000000001 = 1      (bit 0 set)
1 << 1 = 0000000000000010 = 2      (bit 1 set)
1 << 2 = 0000000000000100 = 4      (bit 2 set)
```

```

1 << 3 = 0000000000001000 = 8      (bit 3 set)
1 << 5 = 0000000000100000 = 32     (bit 5 set)
1 << 15 = 1000000000000000 = 32768 (bit 15 set)

```

Each result has exactly one bit set — a “1-hot” value. If we XOR this with the codeword, it flips exactly that one bit. That’s the correction.

But we also need to account for `p_all`. If `p_all = 0` (no error or double error), we want the mask to be all zeros (flip nothing). So the full mask we want is:

```
EVIDENCE = p_all << syndrome
```

- If `p_all = 0`: `EVIDENCE = 0` regardless of syndrome. No correction.
- If `p_all = 1`: `EVIDENCE = 1 « syndrome`. Corrects the bad bit.

We call this value **EVIDENCE** because it represents the evidence of where the error is — a single-bit “pointer” to the corrupted position. When `EVIDENCE` is 0, there’s no evidence of a correctable error.

The barrel shifter: building `1 « syndrome` from its binary digits

The syndrome is 4 bits: `s3 s2 s1 s0`. As a number:

```
syndrome = s0 + 2*s1 + 4*s2 + 8*s3
```

```
So 1 << syndrome = 1 << (s0 + 2*s1 + 4*s2 + 8*s3).
```

Left-shifting decomposes over addition:

```
1 << (a + b) = (1 << a) << b
```

So we can build the shift in stages:

Start with: 1

```

Shift left by s0 * 1 → 1 << s0      (shift by 0 or 1)
Shift left by s1 * 2 → above << (s1*2) (shift by 0 or 2)
Shift left by s2 * 4 → above << (s2*4) (shift by 0 or 4)
Shift left by s3 * 8 → above << (s3*8) (shift by 0 or 8)

```

Each stage is a **conditional shift**: shift by 2^i if `s_i = 1`, else don’t shift. This is exactly what a barrel shifter does in hardware.

Actually we start with `p_all` (0 or 1) instead of 1, so the full construction is:

```

EVIDENCE = p_all      (0 or 1)
if s0: EVIDENCE <=< 1  (conditional shift by 1)
if s1: EVIDENCE <=< 2  (conditional shift by 2)
if s2: EVIDENCE <=< 4  (conditional shift by 4)
if s3: EVIDENCE <=< 8  (conditional shift by 8)

```

If `p_all = 0`, every shift operates on 0, so `EVIDENCE` stays 0 throughout. If `p_all = 1`, the shifts build up `1 << syndrome`.

How fb2d implements conditional rotation

fb2d has these relevant opcodes:

- **l**: rotate [H0] left by 1 bit (circular, so bit 15 wraps to bit 0)
- **r**: rotate [H0] right by 1 bit
- **f**: if bit 0 of [CL] is 1, swap [H0] and [H1] (Fredkin gate)

We use two cells: **EV** (EVIDENCE, in [H0]) and **SCR** (SCRATCH, in [H1], starts at 0).

The pattern for one barrel stage (conditional rotate left by 2^i , gated on syndrome bit s_i):

CL points to S_i cell (whose bit 0 = s_i)

```
l × (2i)      rotate EV left by 2i
f              if si=1: swap EV  SCR
r × (2i)      rotate [H0] right by 2i
f              if si=1: swap EV  SCR
```

If $s_i = 0$ (don't shift): - **l** rotates EV left. **f** does nothing (bit 0 is 0). **r** rotates EV right by the same amount. **f** does nothing. Net effect: EV unchanged, SCR unchanged.

If $s_i = 1$ (do shift): - **l** rotates EV left by 2^i . Call the result EV'. - First **f**: swaps EV' into SCR, puts 0 (old SCR) into EV cell. - **r** rotates [H0] = 0 right. Still 0. - Second **f**: swaps back. EV gets EV' (the rotated value), SCR gets 0. - Net effect: EV rotated left by 2^i . SCR back to 0.

The key insight: **SCR is always 0 after each stage**, regardless of whether the shift happened. This is what makes it reversible and clean.

(Note: fb2d uses circular rotation, not shift, because cells are 16-bit. But for values with only one bit set, rotation and shift are equivalent for shift amounts 0-15.)

The full algorithm, phase by phase

Setup

Three-row torus:

```
Row 0 (DATA):  CW (the codeword to correct)
Row 1 (CODE):  [336 opcodes...]
Row 2 (GP):    PA  S0  S1  S2  S3  EV  SCR  ROT
                0   1   2   3   4   5   6   7
```

GP row cells all start at 0. Heads: H0 and H1 on CW, CL on ROT (payload 0), GP on PA.

Phase A: Compute overall parity (~32 ops)

Move H0 to PA. Use Y (fused rotate-XOR) at 16 different rotation amounts to XOR all 16 bits of CW into PA.bit0.

After: PA.bit0 = p_all. (Other bits of PA contain Y-accumulated junk.)

Phase B: Extract p_all into EVIDENCE (~6 ops)

Move H0 to EV. Use z to swap bit 0 of EV with bit 0 of [GP] (= PA).

After: EV = p_all (raw 0 or 1). PA.bit0 = 0.

Phase A': Uncompute PA (~36 ops)

Re-run the same Y operations in reverse to cancel the junk in PA. Since bit 0 was changed by z, the uncompute leaves PA = p_all (the junk cancels but the z modification remains). CL returns to payload 0.

After: PA = raw p_all (0 or 1). All other PA bits clean.

Phase C: Compute syndrome (~84 ops)

Standard Hamming syndrome computation. For each syndrome bit s_i, use Y to XOR the relevant bit positions of CW into accumulator S_i.

After: S_i.bit0 = s_i. (Other bits contain Y junk, but we only need bit 0.)

Phase D: Barrel shifter (~55 ops)

H0 on EV (currently p_all), H1 on SCR (= 0). For each i = 0,1,2,3:

Move CL to S_i

$l \times (2^i), \quad f, \quad r \times (2^i), \quad f$

After all 4 stages: EV = p_all « syndrome.

Phase C': Uncompute syndrome accumulators (~94 ops)

Re-run the same Y operations in reverse to clean S0-S3 back to 0. CL returns to payload 0.

After: S0 = S1 = S2 = S3 = 0 (clean). SCR = 0 (never got dirty). ROT = 0 (CL restored it).

Phase E: Correction (~9 ops)

Move H0 to CW, H1 to EV. Execute x (XOR): $CW \hat{=} EVIDENCE$.

This flips the single bit identified by the syndrome, or does nothing if EVIDENCE = 0.

Phase F: Cleanup (~13 ops)

We now have two potentially dirty cells: PA and EV. We merge them into at most one dirty cell using $z + x$:

```
z: swap bit 0 of EV with bit 0 of PA (via GP)
x: EV ^= PA
```

See the worked example below for why this works.

Epilogue (~7 ops)

Move H0 and H1 back to CW. All heads restored.

Total (standalone gadget): 336 ops.

H2 Copy-Down Variant (dual-gadget mutual correction)

In the dual-gadget architecture, two gadgets correct each other's code. Each gadget's H2 scans the other gadget's code cells. Instead of H0 shuttling between the data row and GP row, a **copy-down pattern** keeps all correction work local to the GP row ("stomach"):

Stomach layout (9 slots):

EV	PA	CWL	S0	S1	S2	S3	SCR	ROT
0	1	2	3	4	5	6	7	8

- **CWL** (codeword local): local copy of the remote code cell
- **EV** (evidence): correction mask ($p_all \ll$ syndrome)
- **PA** (parity): overall parity accumulator
- **S0-S3**: syndrome bit accumulators
- **SCR** (scratch): barrel shifter scratch cell (always 0 after use)
- **ROT** (rotation): CL sits here; used for rotation counting

The flow becomes:

```
Copy-in:    m                                - [H0] ^= [H2] (CWL gets remote CW)
Phase A:    Y parity
Phase B:    z-extract p_all → EV
Phase A':   Y-uncompute PA
Phase C:    Y syndrome
Phase D:    Barrel shifter → EV = p_all << syndrome
Phase C':   Y-uncompute S0-S3
Uncompute:  M                                - payload(H0) -= payload(H2) (CWL → 0)
Write-back: j                                - [H2] ^= [H0] (remote gets mask XOR)
Phase F:    z+x cleanup
```

After Phase F: H0=EV, H1=PA, GP=EV. At most 1 dirty cell (EV or PA).

Total (copy-down): 323 ops (315 correction + 3 epilogue + 5 advance).

Contained Design: Phase G (waste cleanup)

In the **contained serpentine** design, H0/H1/CL/GP stay at fixed positions on the stomach row across cycles. Only H2 advances (serpentine scan). The “advance” block ($E \oplus e \rightarrow$) is removed. This means waste from EV and PA can’t be left behind as heads advance — it must be deposited into a **dump corridor** on the same row.

Stomach + dump corridor layout

```
EV PA CWL S0 S1 S2 S3 SCR ROT | dump9 dump10 ... dump13 dump14
0  1  2  3  4  5  6  7  8  | 9    10        13    14
<----- stomach ----->  <--- dump corridor --->
```

Dump slots 9-13 hold EV waste; slot 14 holds PA waste.

Phase G: f-chain waste deposit (~51 ops)

After Phase F, the stomach has at most two dirty cells: EV (with evidence mask) and PA (with p_all). All waste values have **bit 0 = 1**.

Phase G deposits this waste into the dump corridor using f (bit-0 Fredkin gate: if $[CL] \& 1$, swap $[H0] \ [H1]$):

```
T                EV waste → ROT (CL's cell). CL.bit0 = waste & 1.
E × 8            H0: EV → ROT
e × 8            H1: PA → dump9
f e f e f e f e f 5 conditional swaps (shift-register deposit)
] K T K [        K-T-K trick: PA waste → ROT via CL/GP register swap
e f              Single f deposits PA waste at dump14
W × 6            H0: ROT → CWL
w × 12           H1: dump14 → CWL
```

Clean cells (EV=0, PA=0): CL.bit0=0 after T, all f ops are no-ops. No cell values change. No drift.

Dirty cells: CL.bit0=1, f fires. The f -chain acts as a shift register — each dirty cycle pushes waste one slot deeper into the dump corridor. N_DUMP=5 handles up to 5 dirty cycles before overflow.

Preamble: T x (cycle start cleanup)

Each cycle begins with T x which cleans any residual in ROT:

- T: swap $[CL=ROT] \ [H0=CWL]$. Deposits ROT’s value into CWL.
- x: $[CWL] \hat{=} [CWL] = 0$. Zeros CWL (destroying the deposited value).

This also cleans the handler’s CL++ signal (: sets [ROT]=1, preamble zeros it). The **x** with H0=H1 is intentionally non-injective — it destroys the handler signal, which is acceptable because the signal has no information content beyond “handler ran.”

Waste value summary

Error type	R_EV	R_PA	R_EV.bit0	R_PA.bit0
None	0	0	0	0
Bit 0	0	1	0	1
Bit k (1)	(1«k) 1	0	1	0
Double	0	0	0	0

All nonzero waste values have bit 0 = 1. This is why **f** (bit-0 Fredkin) works but **F** (payload Fredkin, checks DATA_MASK) does NOT — waste values like 0x5 = bits {0,2} have no data bits set and pass through DATA_MASK as zero.

Known issue: **f** reversibility when CL = H0

During the f-chain, H0 and CL both point to ROT. When **f** fires (dirty cycle), it swaps [H0=ROT] [H1=dump], zeroing ROT. This changes [CL] from nonzero to zero, making **f** non-self-inverse: applying **f** again reads [CL]=0 and doesn’t fire. The function is **not injective** when CL=H0 and [H1]=0.

This affects **step_back()** (simulator reverse stepping) but not forward operation. The ouroboros never runs backward. A proper fix requires separating CL from H0 during **f** ops, likely via XOR-copy of the waste flag to a dedicated condition cell. See the head-overlap reversibility discussion below.

Head-overlap reversibility constraints

Several fb2d ops are NOT bijective when heads overlap:

Op	Overlap	Effect	Bijective?
x	H0=H1	[H0] $\wedge$ = [H0] = 0	No (all values $\rightarrow$ 0)
,	H0=H1	[H0] $\neg$ = [H0] = 0	No
m	H0=H2	[H0] $\wedge$ = [H0] = 0	No
j	H2=H0	[H2] $\wedge$ = [H2] = 0	No
f/F	CL=H0 or CL=H1	Condition changes with swap	No (when [other]=0)

All other ops (swaps: X, T, V, Z, K; increments: +, -, :, ;; rotations: r, l, R, L; mirrors) are bijective under all head configurations.

For the language to be truly reversible (“every state has a unique predecessor”), programs must avoid these overlaps during non-injective ops, OR the ops must be redefined to be identity (NOP) under overlap. The preamble’s x with H0=H1 is an intentional information-destroying use (zeroing a cell whose value is known/discardable).

Worked example: payload 42, error on bit 5

Encoding

Payload 42 = 0b000000101010. Hamming encodes to codeword 0x05A0 = 0000 0101 1010 0000.

Bit positions (standard form, bit 15 at left):

Position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit:	0	0	0	0	0	1	0	1	1	0	1	0	0	0	0	0

~
bit 5 (data bit)

Error injection

Flip bit 5: codeword becomes 0x0580 = 0000 0101 1000 0000.

Position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Original:	0	0	0	0	0	1	0	1	1	0	1	0	0	0	0	0
Corrupted:	0	0	0	0	0	1	0	1	1	0	0	0	0	0	0	0

~
flipped!

Phase A result

p_all = XOR of all 16 bits of corrupted codeword = XOR of {0,0,0,0,0,1,0,1,1,0,0,0,0,0,0,0}
= 1.

(An odd number of bits are set, meaning an odd number were flipped from the valid codeword which always has even weight.)

Phase B result

EV = 1 (raw). PA.bit0 = 0.

Phase A’ result

PA = 1 (raw p_all).

Phase C result

Syndrome computation — each s_i is the XOR of specific bit positions:

- s_0 = XOR of positions $\{1,3,5,7,9,11,13,15\} = 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0 = 1$
- s_1 = XOR of positions $\{2,3,6,7,10,11,14,15\} = 0\ 0\ 0\ 1\ 1\ 0\ 0\ 0 = 0$
- s_2 = XOR of positions $\{4,5,6,7,12,13,14,15\} = 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0 = 1$
- s_3 = XOR of positions $\{8,9,10,11,12,13,14,15\} = 1\ 0\ 1\ 0\ 0\ 0\ 0\ 0 = 0$

Syndrome = 0b0101 = 5. Correct! The error is at position 5.

Phase D: barrel shifter

EV starts as 1 (= p_{all}). Syndrome bits: $s_0=1$, $s_1=0$, $s_2=1$, $s_3=0$.

Stage 0 ($s_0=1$, shift by 1):

```
EV = 1 = ...0001
l×1:      ...0010    (rotated left 1)
f: swap EV SCR ( $s_0=1$ )
r×1: SCR=0, rotate 0, still 0
f: swap back
Result: EV = ...0010 = 2
```

Stage 1 ($s_1=0$, shift by 2):

```
EV = 2 = ...0010
l×2:      ...1000    (rotated left 2)
f: no swap ( $s_1=0$ )
r×2:      ...0010    (rotated right 2, back to original)
f: no swap
Result: EV = ...0010 = 2    (unchanged)
```

Stage 2 ($s_2=1$, shift by 4):

```
EV = 2 = ...0000 0010
l×4:      ...0010 0000    (rotated left 4)
f: swap EV SCR ( $s_2=1$ )
r×4: SCR=0, still 0
f: swap back
Result: EV = ...0010 0000 = 32
```

Stage 3 ($s_3=0$, shift by 8):

```
(same as stage 1 -  $s_3=0$ , so l and r cancel)
Result: EV = 32    (unchanged)
```

Final: $EV = 32 = 0000\ 0000\ 0010\ 0000$ = bit 5 set. This is $1 \ll 5$. Exactly the mask we need!

Phase E: correction

$CW \hat{=} EV$: flips bit 5 of the corrupted codeword, restoring it to the original 0x05A0. Correction complete!

Phase F: cleanup

Before cleanup: $PA = 1$, $EV = 32 = \dots 0010\ 0000$.

z: swap bit 0 of EV with bit 0 of PA (via GP).

EV.bit0 was 0, PA.bit0 was 1.

After: $EV = 33 = \dots 0010\ 0001$, $PA = 0$.

x: $EV \hat{=} PA = 33 \hat{=} 0 = 33$.

(PA is 0, so XOR changes nothing.)

Result: $PA = 0$ (clean!), $EV = 33$ (dirty). **One dirty cell.**

Why the cleanup works in all cases

The $z+x$ trick handles every case:

No error ($PA=0$, $EV=0$): z swaps two 0 bits. x XORs with 0. Both stay 0. **0 dirty cells.**

Double error ($PA=0$, $EV=0$): Same as no error — $p_all=0$ means EVIDENCE was never nonzero. **0 dirty cells.**

Bit-0 error ($PA=1$, $EV=1$): z: swap bit 0 of $EV(=1)$ with bit 0 of $PA(=1)$. Both are 1, so no change. $EV=1$, $PA=1$. x: $EV \hat{=} PA = 1 \hat{=} 1 = 0$. PA still 1. Result: $EV=0$ (clean), $PA=1$ (dirty). **1 dirty cell.**

Bit-k error, k 0 ($PA=1$, $EV=1 \ll k$): z: swap bit 0 of $EV(=0, \text{ since } k \neq 0)$ with bit 0 of $PA(=1)$. EV gains a 1 in bit 0: $EV = (1 \ll k) | 1$. PA loses its bit 0: $PA=0$. x: $EV \hat{=} PA = EV \hat{=} 0$, no change. Result: $PA=0$ (clean), $EV=(1 \ll k) | 1$ (dirty). **1 dirty cell.**

In every case, at most one cell is nonzero. The dirty value in EV is not a valid Hamming codeword — it has nonzero syndrome, which is how you can tell it's garbage rather than data.

Reversibility

Everything is reversible. Running `step_back()` through all 336 steps restores the grid to its exact original state: corrupted codeword back in CW, all GP cells back to 0. The dirty cell from Phase F gets cleaned by the reverse of $z+x$, the Y-uncompute phases re-dirty and then un-dirty the accumulators, and so on.

Cost summary

Variant	Code ops	Dirty cells (error)	Dirty cells (clean)	Stomach width
Standalone	336	1	0	8 cells
H2	323	1	0	9 cells
copy- down				
Contained	372	1 (in dump corridor)	0	9 + N_DUMP+1 cells
serpen- tine				

Each correction deposits at most 1 waste cell into the dump corridor (for errors) or 0 (for clean codewords). Clean zeros are the agent's fundamental resource — this is about as cheap as correction can get.